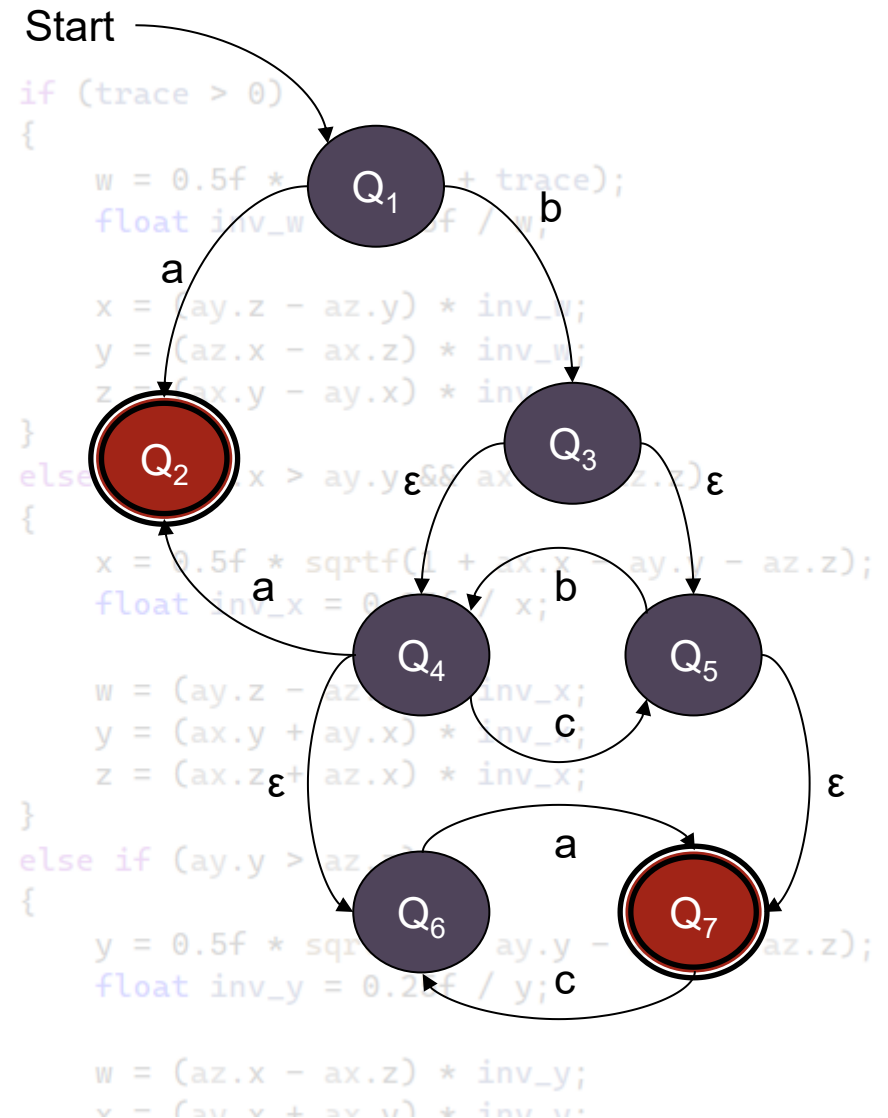


Algorithmen und Datenstrukturen

2 – Grundlagen C und C++

Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de



Einleitung

Heranführung

- Was ist die beste Programmiersprache? Warum?
- Was macht ein Compiler?
- Was ist das abstrakte Speichermodell?
- Was sind virtuelle Adressen?
- Wie lange existiert eine Variable?

Kompetenzen

- Überblick über Programmiersprachen gewinnen
- C-Programmierung wiederholen und festigen:
 - Quellcodedateien, Programmaufbau, Zeiger, Stack und Heap
- einfache Speicherverwaltung in C++ vornehmen

Vortragsübersicht

2 – Grundlagen C und C++

- C und C++ im Überblick
- Compiler-Pipeline
- Programmaufbau
- Werte und Zeiger
- Stack und Heap

C und C++ im Überblick (1)

Wichtige Bewertungskriterien für Programmiersprachen

- **Effizienz** der fertigen Programme
 - Wie schnell und stromsparend sind die Programme?
 - Machen die Programme unnötige Berechnungen?
- **Robustheit** der fertigen Programme
 - Wie schlimm ist es, wenn ich einen Fehler einprogrammiere?
 - Werden Fehler möglichst früh sowie zuverlässig erkannt?
- **Skalierung** für komplexe Aufgaben
 - Wie gut sind im Kern neue, komplizierte Programme umzusetzen?
- **Ansprüche** an Programmierer
 - Wie schwer ist die Sprache zu erlernen?
 - Wie viele Spezialfälle und Stolpersteine gibt es?
- **Hacking-Faktor**
 - Wie viel Zeit und Code brauche ich, um Programme zu erstellen?

C und C++ im Überblick (2)

Qualitative Bewertung einiger, typischer Programmiersprachen

Sprache	Effizienz	Robustheit	Skalierung	Ansprüche	Hacking
Assembler	optimal	schlecht	miserabel	mittel	miserabel
C	sehr gut	schlecht	mittelmäßig	mittel	schlecht
C++	sehr gut	miserabel	gut	sehr hoch	mittelmäßig
python	miserabel	mittelmäßig	mittelmäßig	gering	sehr gut
Java	mittelmäßig	sehr gut	sehr gut	gering	gut
C#	mittelmäßig	sehr gut	sehr gut	gering	gut

- C ist besonders sinnvoll, wenn es mittelmäßig komplexe Aufgaben zu lösen gilt, bei denen es auf Geschwindigkeit ankommt (z. B. Bildverarbeitung ohne KI)
 - C++ ist besonders sinnvoll, wenn Aufgaben mit komplexer Struktur zu lösen sind, bei denen es trotzdem auf Geschwindigkeit ankommt (z. B. Computerspiele)
- **in allen anderen Fällen sind andere Sprachen die bessere Wahl!**

C und C++ im Überblick (3)

C vor der Standardisierung



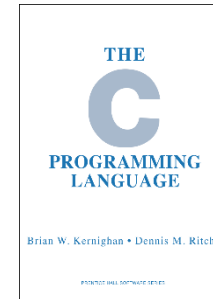
ab 1970:
Dennis Ritchie (rechts)
und Ken Thompson (links)
entwickeln im Rahmen
ihrer Arbeit an *UNIX* bei
den Bell Labs die Sprache *B*
als vereinfachte Variante
von *BCPL* (von 1967) und
ALGOL (von 1965)

Erste Schritte

ab 1971:
Erweiterung und
Umbenennung zu *C*

ab 1972:
Bestandteil von UNIX Version 2,
zusätzliche Features
(z. B. eigene Datenstrukturen)

ab 1973:
UNIX Version 4 ist fast vollständig
in *C* neu geschrieben



K&R C

1978:
Buch
"The C Programming Language"
als inoffizieller Standard,
herausgegeben von Ritchie
und Thompson
(manchmal auch: C78)

ab 1978:
unabhängige C-Compiler
auf vielen Rechnern vom
Großcomputer bis zum PC
setzen K&R C um

1979: C++ zweigt
von K&R C ab

C und C++ im Überblick (4)

C ab der Standardisierung

ANSI C

1989:
offizielle Standardisierung
durch die *ANSI (American
National Standards Institute)*,
daher manchmal auch C89

1990:
ANSI-Standards durch die
*ISO (International Organization
for Standardization)* übernommen,
daher manchmal auch C90

C99

1999:
neuer ISO-Standard
ISO/IEC 9899:1999 mit
einigen Zusatzfeatures,
z. B.:
zusätzliche Datentypen,
einzeilige Kommentare

C11

2011:
neuer ISO-Standard
ISO/IEC 9899:1999 mit
vielen Änderungen,
z. B.:
generische Makros,
UNICODE-Support,
Multi-Threading und
atomare Operationen

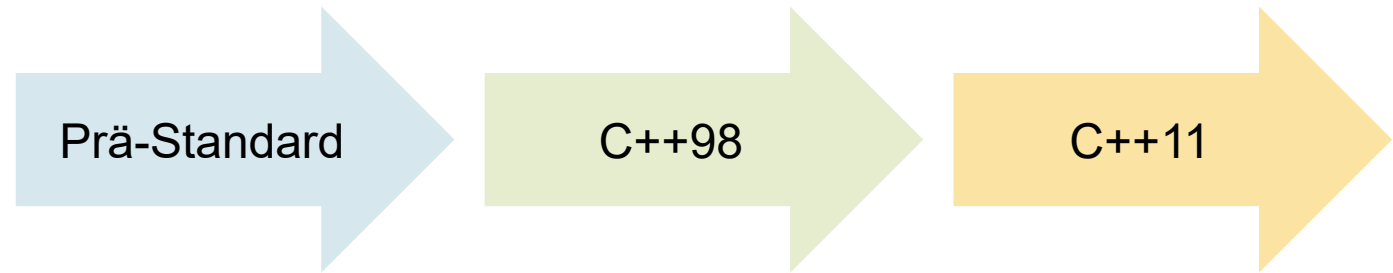
C23

2023:
nächster ISO-
Standard geplant



C und C++ im Überblick (5)

C++ in großen Sprüngen



ab 1979:
Bjarne Stroustrup
entwickelt bei AT&T
"C mit Klassen"

ab 1983:
Name C++

1985:
Referenz-Compiler

1989:
Mehrfachvererbung
const, static, protected

1991:
Templates

ISO-Standard
ISO/IEC 14882:1998
inklusive STL,
Standard Template
Library

auto
-> return_type
initializer_list

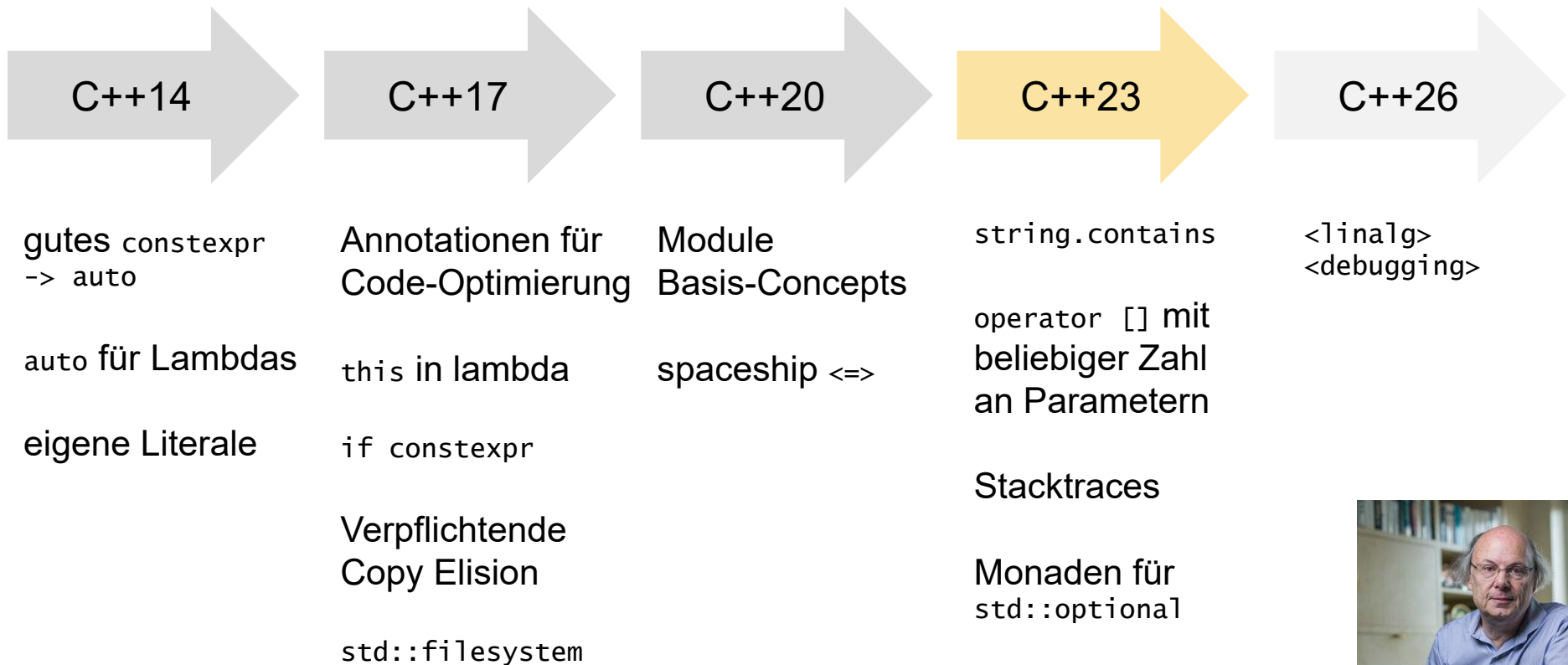
rvalue und move
Lambdas
Ausnahmesicherheit

größere STL
z. B. regexp, Threads

Sprachzucker

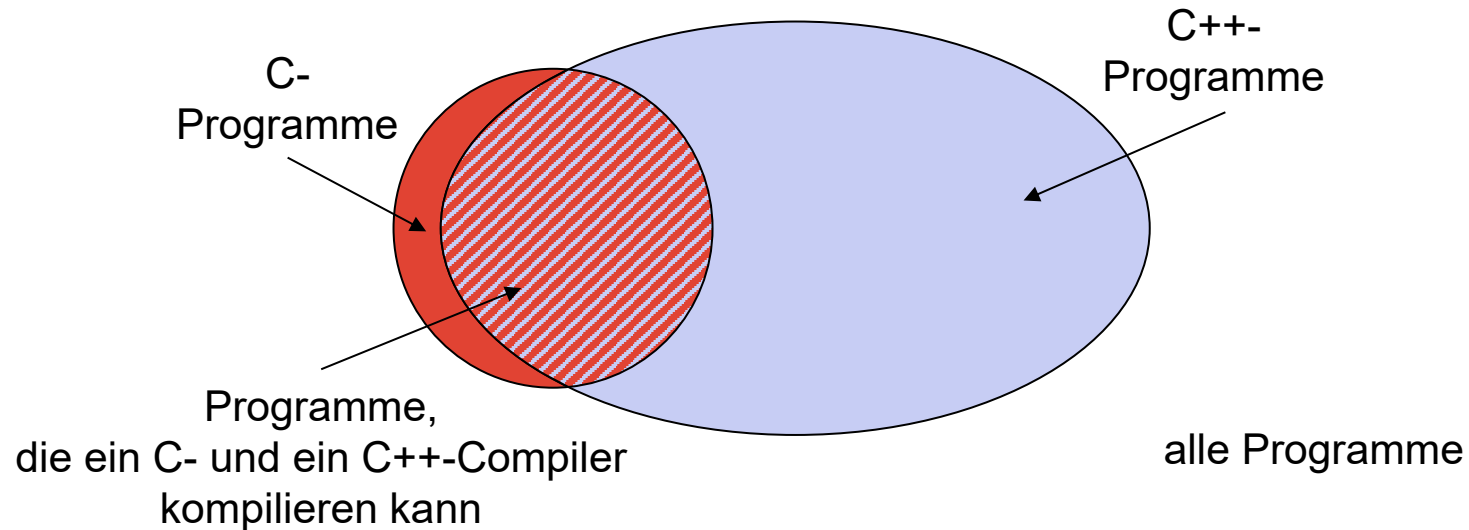
C und C++ im Überblick (6)

C++ in kleinen Schritten



C und C++ im Überblick (7)

Das Verhältnis zwischen C und C++



- qualitative Mengendarstellung im *Venn-Diagramm* (auch *Mengendiagramm*)
- jeder Punkt im Diagramm ist ein theoretisch mögliches Programm
 - ein großer Teil der C-Spezifikation ist in der C++-Spezifikation nachgebaut (die Spezifikationen sind aber grundsätzlich unabhängig!)
 - in der Praxis kann ein C++-Compiler auch C-Quellcode übersetzen

C und C++ im Überblick (8)

Gegenüberstellung C und C++ am Beispiel

// Programm in ANSI C

```
#include <stdio.h>
```

```
int main(int argc, char** argv)
{
    int z;
    for (z = 0; z < 10; ++z)
        printf("%i", z);
}
```

// Programm in C++

```
#include <iostream>
```

```
int main(int argc, char** argv)
{
    for (int z = 0; z < 10; ++z)
        std::cout << z;
}
```

- das linke Programm ist im ANSI-C-Syntax geschrieben
→ das Programm kompiliert sowohl in C- als auch in C++-Compilern
- das rechte Programm ist im C++98-Syntax geschrieben
→ das Programm kompiliert nur in C++-Compilern

C und C++ im Überblick (9)

Von C zu C++

- oberflächlich kaum relevante Änderungen
 - Funktionen (inklusive Deklaration und Definition) wie bekannt
 - Variablen (z. B. `int`, `float`) wie bekannt
 - Quellcode-Dateien haben jetzt `.cpp` als Dateiendung
 - Header-Dateien haben jetzt `.hpp` (oder `.h`) als Dateiendung
- im Detail problematische Abweichungen
 - C++ ist **deutlich mehr** als nur "C mit Klassen"
 - Ignorieren von C++-Features (z. B. Ausnahmen) erzeugt subtil kaputten Code

Beispiel:

```
void insert_into_bag
(std::list<item> bag, item a, item b, item c)
{
    bag->insert(a);
    bag->insert(b);    // C++: falls Fehler, bleibt a in bag, c wird nie eingefügt
    bag->insert(c);    // → in C wäre das nur mit Tricks (setjmp) möglich
}
```

C und C++ im Überblick (10)

Ausgewählte Sprach-Features in C und C++

Feature	C	C++
Paradigmen	imperativ, strukturiert, prozedurell	imperativ, strukturiert, prozedurell, generisch, objektorientiert, funktional
Typsystem	statische Typisierung, statische Prüfung	statische Typisierung, härtere, statische Prüfung, optionale, dynamische Prüfung
Speicherverwaltung	manuell, maschinennah	manuell, maschinennah, optional rudimentäre, automatische Speicherverwaltung
eigene Datentypen	nur Wertesammlungen	Klassen mit Objekten, Methoden, Kapselung, Mehrfach-Vererbung
Standardbibliothek	einfache Hilfsfunktionen (z. B. keine String-Objekte) und Systemfunktionen	alles aus C, zudem höhere Klassen (z. B. Strings, Threads)
Generischer Code	Makros mit rein-textueller Ersetzung	compiler-geprüfte Compile-Zeit-Vorlagen (templates)
Funktionale Programmierung	nur über Umwege	über Bibliothek integriert

C und C++ im Überblick (11)

Wie man C++ lernt

Ziel:



C und C++ im Überblick (12)

Problem:



<https://netzpolitik.org>

Vortragsübersicht

2 – Grundlagen C und C++

- C und C++ im Überblick
- Compiler-Pipeline
- Programmaufbau
- Werte und Zeiger
- Stack und Heap

Compiler-Pipeline (1)

Allgemeine Programmstruktur

- ein Programm einer höheren Programmiersprache besteht üblicherweise aus *Funktionen* und *Datenstrukturen*
- Funktionen und Datenstrukturen gliedern sich üblicherweise in *hierarchische Einheiten* (z. B. *Module*, *Pakete*, *Bibliotheken*, *Assemblies*, *Übersetzungseinheiten*)
- zwischen Einheiten entstehen *Abhängigkeiten*, wenn eine Einheit Funktionen oder Datenstrukturen einer anderen Einheit verwendet

Beispiel in C#:

Assembly `worldModel.dll`
enthält CIL-Code für

```
namespace worldModel
{
    class Robot      { /* ... */ }
    class Obstacle  { /* ... */ }
}
```

Assembly `userInterface.dll`
enthält CIL-Code für

```
class MainWindow
{
    void Draw(Obstacle o) { /*... */ }

    /* ... */
}
```

Abhängigkeit



Compiler-Pipeline (2)

Programmstruktur in C++

- *Übersetzungseinheit* (oder *Modul*, d. h. eine Quellcodedatei) `world_model.cpp` hat die Klassen `robot` und `obstacle` mit Variablen und Funktionen

Übersetzungseinheit `world_model.cpp`

```
class robot { void goto(const waymark& w); /* ... */ };  
class obstacle { int x; int y; image get_graphics(); /* ... */ };
```

- Übersetzungseinheit `user_interface.cpp` hat die Klasse `main_window` und Funktionen zum Zeichnen eines Hindernisses sowie zur Darstellung eines Bildes am Bildschirm

Übersetzungseinheit `user_interface.cpp`

```
class main_window  
{ void draw_obstacle(const obstacle& o);  
  void copy_to_screen(const image& i, int x, int y); /* ... */ };
```

→ die Übersetzungseinheit `user_interface.cpp` hängt von der Übersetzungseinheit `world_model.cpp` ab

ACHTUNG:
Beispiele sind noch kein
vollständig korrektes C++!

Compiler-Pipeline (3)

Teilschritte der Programmerstellung (*Bauen*, engl. *building*)

Kompilieren (Übersetzen) durch *Compiler*:

- Einheiten (Quellcodedateien, Pakete) werden separat übersetzt
- Abhängigkeiten werden üblicherweise semantisch geprüft
- Ergebnis ist ein *Programmfragment* pro Übersetzungseinheit

Linken (Verknüpfen) durch *Linker*:

- Programmfragmente werden zu einem Programm zusammengebunden
- Abhängigkeiten werden üblicherweise inhaltlich aufgelöst
- der Linker kann ein separates Programm oder Teil des Compilers sein

Beispiel:

```
void main_window::  
    draw_obstacle(const obstacle& o)  
{  
    image i(o.get_graphics());  
    copy_to_screen(i, o.x, o.y);  
}
```

nach Kompilieren:

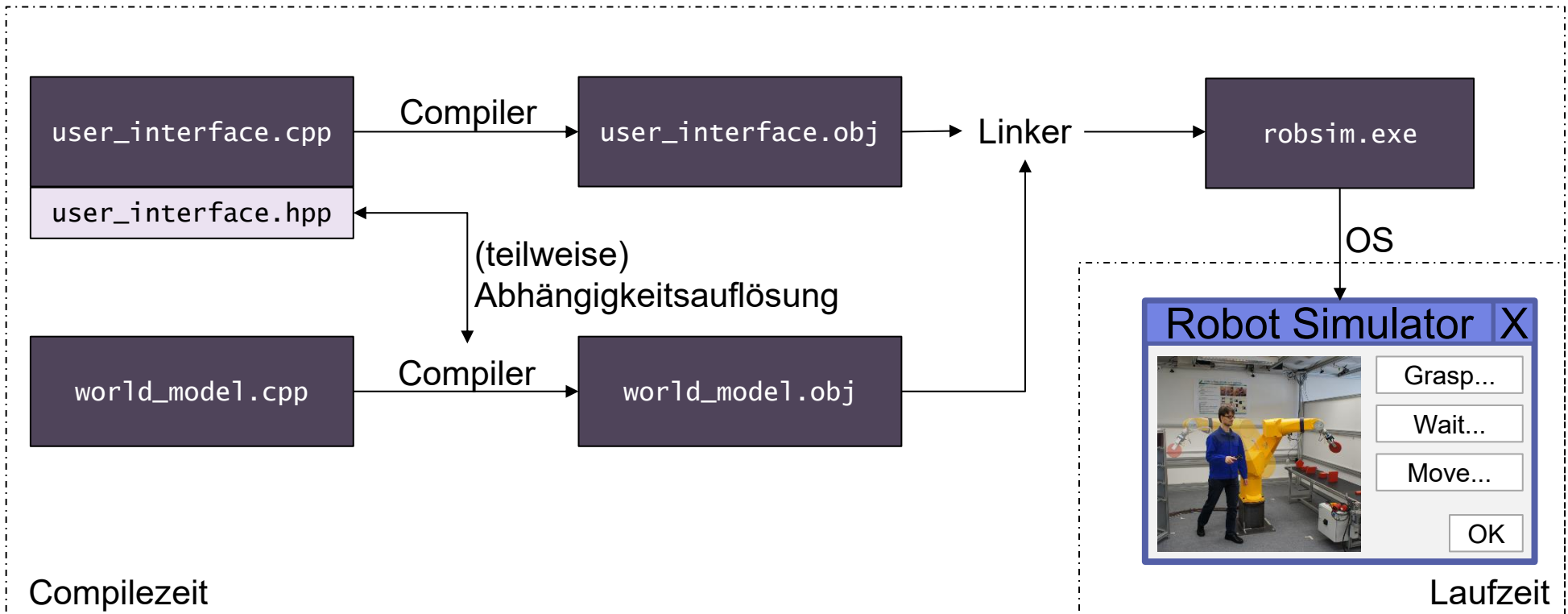
...
ref o.get_graphics()
...
call 0x00219aef

nach Linken:

...
call 0x00133bc9
...
call 0x00219aef

Compiler-Pipeline (4)

Vom Code zum Programm in C



- modulweise Kompilierung zu Maschinencode (*object files*, z. B. Endung `.obj`)
- Abhängigkeitsprüfung (und teilweise Auflösung) beim Kompilieren (`.hpp` → später mehr!)
- vollständige Abhängigkeitsauflösung beim Linken

Compiler-Pipeline (5)

Manuelles Kompilieren

manueller Aufruf der Compiler-Werkzeuge über die Kommandozeile

- Eingabe aller .c-Dateien in separate Compiler-Aufrufe

z. B.

```
cl user_interface.cpp -o user_interface.obj
```

```
cl world_model.cpp -o world_model.obj
```

- Eingabe aller entstandenen .obj-Dateien in den Linker

z. B.

```
lnk user_interface.obj world_model.obj -o robsim.exe
```

- Programm `robsim.exe` ausführen

Bewertung:

- aufwendig
- fehleranfällig
- lange Kompilierzeit

Compiler-Pipeline (6)

Automatisches Kompilieren

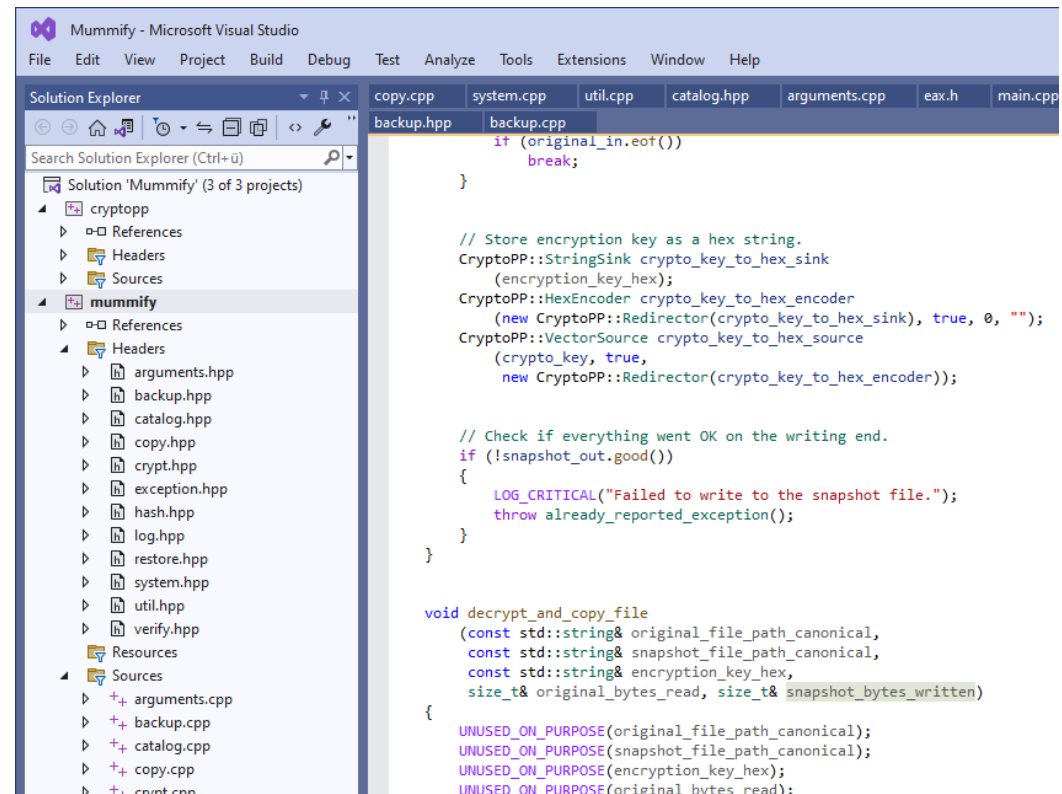
- über Skriptsprache (z. B. makefile, CMake) oder über IDE
- automatisches Erfassen aller Quellcode-Dateien (z. B. mittels drag & drop)
- Kompilieren und Linken nur von geänderten Eingabedateien

Bewertung:

- komfortabel
- fehlertolerant
- schnell

aber:

- die IDE verwendet intern auch nur die Kommandozeilentools
- manchmal wichtig zu wissen (z. B. Umgebungsvariablen müssen korrekt gesetzt sein)



Vortragsübersicht

2 – Grundlagen C und C++

- C und C++ im Überblick
- Compiler-Pipeline
- Programmaufbau
- Werte und Zeiger
- Stack und Heap

Programmaufbau (1)

Quellcode-Dateien in C und C++

- *Source-* oder *Quellcode*dateien (.c / .cpp) mit Implementierung
 - *Header*dateien mit Schnittstellenspezifikation (.h / .hpp) für semantische Prüfung und Teilauflösung von Abhängigkeiten beim Kompilieren
- Achtung: manchmal bezeichnet man Header ebenfalls als Quellcode

Beispiel:

geo_2d.hpp

```
/**
 * @class vector2i
 * @ingroup geo_module
 *
 * A 2D integer vector.
 */
class vector2i
{
public:
    vector2i();
    vector2i(int x, int y);

    const int& operator [](int index) const;
    int& operator [](int index);

    /** Usual vector arithmetics. */
    //@{
    vector2i operator -() const;

    vector2i operator +(const vector2i &v2) const;
    vector2i operator -(const vector2i &v2) const;
    vector2i operator *(const vector2i &v2) const;
    vector2i operator /(const vector2i &v2) const;
```

geo_2d.cpp

```
vector2i::vector2i() : x(0), y(0) { }
vector2i::vector2i(int x, int y) : x(x), y(y) { }

const int& vector2i::operator [](int index) const
{ return c[index]; }
int& vector2i::operator [](int index)
{ return c[index]; }

vector2i vector2i::operator -() const
{ return vector2i(-x, -y); }

vector2i vector2i::operator +(const vector2i &v2) const
{ return vector2i(x + v2.x, y + v2.y); }
vector2i vector2i::operator -(const vector2i &v2) const
{ return vector2i(x - v2.x, y - v2.y); }
vector2i vector2i::operator *(const vector2i &v2) const
{ return vector2i(x * v2.x, y * v2.y); }
vector2i vector2i::operator /(const vector2i &v2) const
{ return vector2i(x / v2.x, y / v2.y); }
```

Programmaufbau (2)

Inhalt der Headerdatei

- Klassennamen
- Klassenmember
- Signaturen für Klassenmethoden und Instanzmethoden
- Signaturen für freie Methoden
- Namensräume
- Typdefinitionen

Beispiel:

```
#ifndef INCLUDED__ROBOSIM__WORLD_MODEL_HPP
#include <list>

namespace robosim
{
    class obstacle;
    typedef std::list<obstacle> obstacle_list;

    class obstacle {
    public:
        obstacle
            (int x, int y);

        static obstacle_list load
                                   (const string& file);
        bool is_interactable() const;

    private:
        const int x_;
        const int y_;
        /* ... */
    };

} /* namespace robosim */
#endif /* INCLUDED__ROBOSIM__WORLD_MODEL_HPP */
```

Programmaufbau (3)

Inhalt der Quellcodedatei

- Namensräume
- Methodenimplementierungen

Faustregel:

**Möglichst viel Inhalt vom Header
in die Quellcodedatei auslagern!**

Beispiel:

```
#include "robosim.hpp"

namespace robosim
{
    obstacle::obstacle
        (int x, int y)
        :
        x_(x), y_(y)
    { }

    obstacle_list obstacle::load
        (const string& file)
    {
        /* ... */
    }

    bool obstacle::is_interactable() const
    {
        return has_buttons_ || has_handle_;
    }

} /* namespace robosim */
```

Programmaufbau (4)

Verwenden von Headerdateien

- Abhängigkeiten mit `#include`-Anweisung einbinden (vgl. `import` anderer Sprachen)

`#include "dateiname.hpp"`

→ für eigene Header des Programms

`#include <dateiname>`

→ für Standardbibliothek

`#include <dateiname.hpp>`

→ für Bibliotheken von Drittanbietern

Beispiel:

```
#include "geo_2d.hpp"
```

```
...
```

```
class image  
{
```

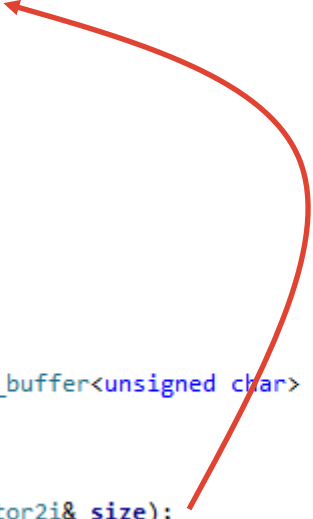
```
public:
```

```
    typedef dynamic_buffer<unsigned char> data_buffer;
```

```
    image();
```

```
    image(const vector2i& size);
```

verwende Klasse `vector2i`
aus Übersetzungseinheit `geo_2d.cpp`
mit Schnittstelle `geo_2d.hpp`



Programmaufbau (5)

Der Präprozessor

- #-Anweisungen werden vom *Präprozessor* (i. d. R. Teil des Compilers) ausgewertet
- `#include` heißt: direktes textuelles Einlesen der angegebenen Datei
- nach Auflösung aller `#includes` zeilenweises Kompilieren von oben nach unten

Beispiel:

```
#include "geo_2d.hpp"
```

```
...
```

```
class image  
{
```

```
public:
```

```
    typedef dynamic_buffer<unsigned char> buffer;
```

```
    image();  
    image(const vector2i& size);
```

kompiliert wird wegen
Präprozessor-Lauf
in Wirklichkeit:

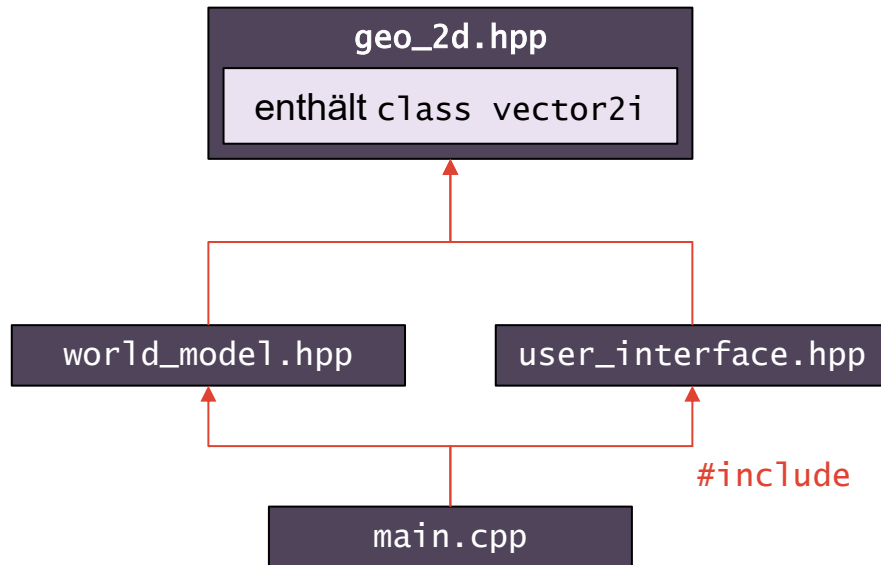


```
/**  
*****  
*  
* @class vector2i  
* @ingroup geo_module  
*  
* A 2D integer vector.  
*  
*****/  
class vector2i  
{  
public:  
    vector2i();  
    vector2i(int x, int y);  
    const int& operator[](int index) const;  
    ...  
}  
  
class image  
{  
public:  
    typedef dynamic_buffer<unsigned char> buffer;  
    image();  
    image(const vector2i& size);
```

Bezeichner
vector2i ist
nun bekannt!

Programmaufbau (6)

Redundante Headereinbindung



- Verfolgen von Abhängigkeiten über mehrere Pfade
- der selbe Header kann (und muss) mehrmals vorkommen

Problem:

der Präprozessor macht aus

```
#include "world_model.hpp"
#include "user_interface.hpp"
```

den Zwischencode

```
#include "geo_2d.hpp"
/* ... */
#include "geo_2d.hpp"
/* ... */
```

und gibt schließlich

```
class vector2i { /* ... */ };
/* ... */
class vector2i { /* ... */ };
```

zum Kompilieren weiter

→ **Fehler beim Kompilieren:**
doppelte Definition der Klasse vector2i

Programmaufbau (7)

Lösung *Inclusion Guards*

Vorgehen:

- setze Präprozessor-Flag bei erstem `#include`
- ignoriere Headerdatei bei weiteren `#include`-Anweisungen
- **eindeutiger Flagname ist enorm wichtig!**
(z. B. nach Schema `INCLUDED__<PROJEKTNAME>__<HEADERNAME>_HPP`)

Beispiel:

```
#if !defined(INCLUDED__VIRAL_CORE__GEO_2D_HPP)
#define INCLUDED__VIRAL_CORE__GEO_2D_HPP

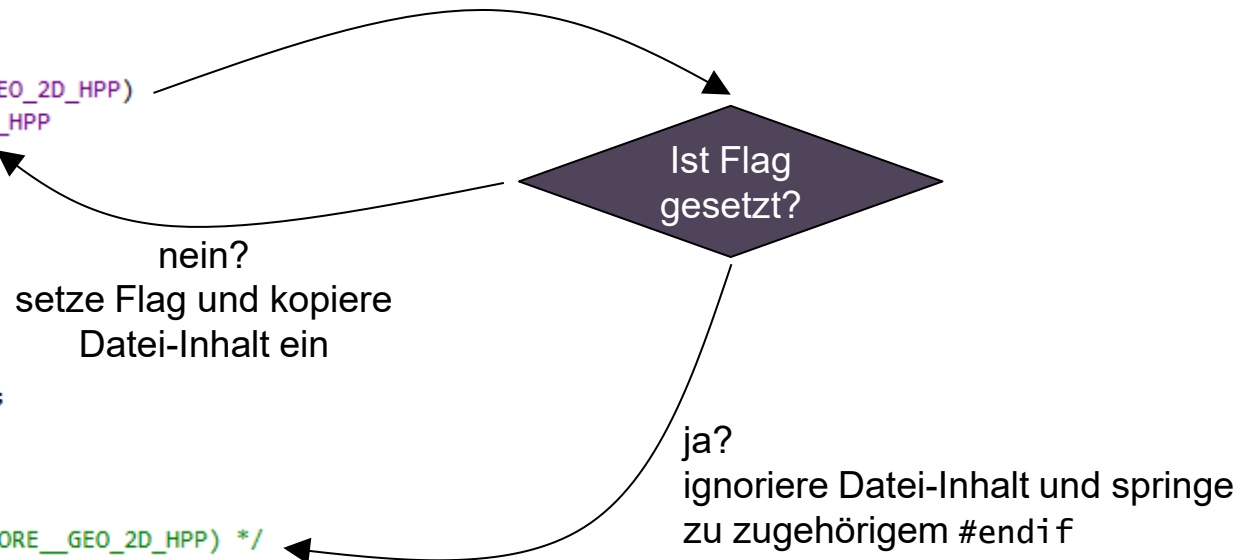
...

class vector2i
{
public:
    vector2i();
    vector2i(int x, int y);

    const int& operator [](int index) const;
    int& operator [] (int index);

...

#endif /* !defined(INCLUDED__VIRAL_CORE__GEO_2D_HPP) */
```



Programmaufbau (8)

Include-Reihenfolge

Headerdatei:

- Standardheader
- Bibliotheken von Drittanbietern
- eigene Header

Beispiel:

```
#if !defined(INCLUDED__ROBOSIM__IMAGE_HPP)
#define INCLUDED__ROBOSIM__IMAGE_HPP

#include <vector>
#include <list>

#include <windows.h>

#include "world_model.hpp"
#include "interface.hpp"
```

Source-Datei:

- zugehörige Headerdatei
- Standardheader
- Bibliotheken von Drittanbietern
- eigene Header

Beispiel:

```
#include "image.hpp"

#include <iostream>

#include <libpng.h>
#include <libjpeg.h>

#include "exceptions.hpp"
#include "logfile.hpp"
```

Begründung → Übungsaufgabe

Vortragsübersicht

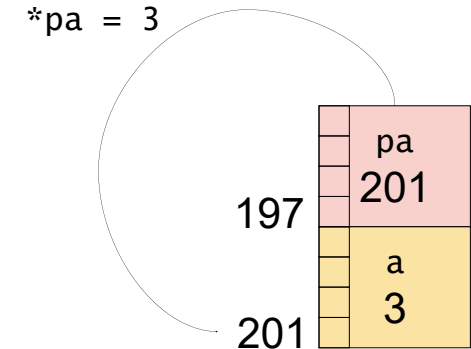
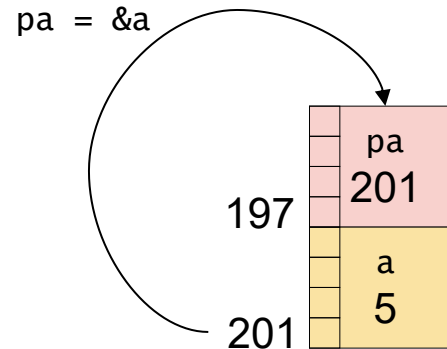
2 – Grundlagen C und C++

- C und C++ im Überblick
- Compiler-Pipeline
- Programmaufbau
- Werte und Zeiger
- Stack und Heap

Werte und Zeiger (1)

Grundlagen für Zeiger

```
void fun_with_pointers()
{
    int a = 5;
    int *pa = &a;
    *pa = 3;
    // a == 3
}
```



- Adresse einer beliebigen Variablen abfragen: `&variablenname`
→ Ergebnis ist temporärer (unbenannter) Zeiger auf diese Variable
- benannte Zeiger-Variable deklarieren: `ziel_datentyp *namederzeigervariablen`
- Zeiger *dereferenzieren* (zum Ziel folgen): `*namederzeigervariablen`

Vorsicht: Syntax!

- der `*` bindet an den Variablennamen (z. B. `int *i`, `j` ist ein Zeiger und ein Integer)
- Leerzeichen nach Belieben (z. B. `int*i`, `int * i`, `int *i`, `int* i` sind alle gleich)

Werte und Zeiger (2)

Dereferenzieren von Zeigern

```
void pointers_and_aliases()
{
    int a = 1;
    int b = 2;

    int *p = &a;
    *p = 3;           // äquivalent zu a = 3 ("Ziel ändern?")

    p = &b;           // biege p auf b um ("Ziel ändern?")
    *p = 4;           // äquivalent zu b = 4
}
```

	p
	201
193	b
	4
197	a
	3
201	

- Ergebnis des Ausdrucks `*namederzeigervariablen` ist identische Variable zur (aktuellen) Zielvariablen des Zeigers
 - überall, wo ich `*namederzeigervariablen` schreibe, könnte ich auch die (aktuelle) Zielvariable direkt hinschreiben (und z. B. ändern)
 - der Ausdruck `*namederzeigervariablen` ist ein *Alias* (anderer Name) für die (aktuelle) Zielvariable des Zeigers

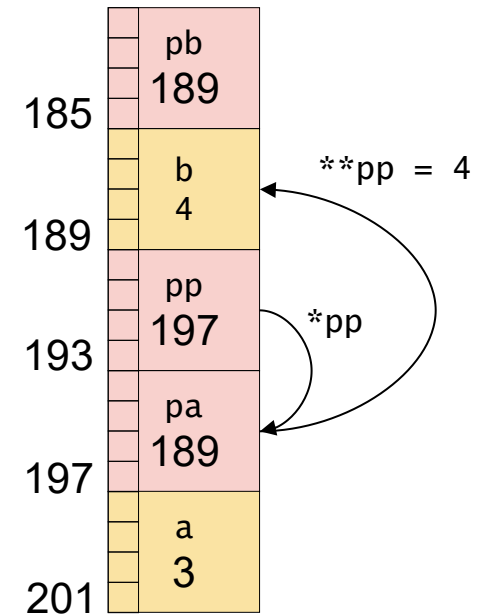
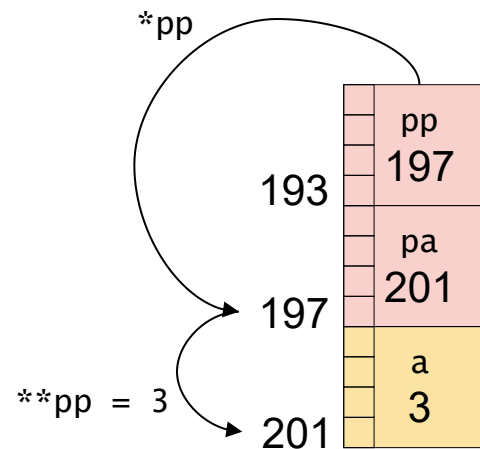
Achtung Formulierung: Was heißt "das Ziel des Zeigers ändern"?

Werte und Zeiger (3)

Mehrstufige Zeiger

```
void multi_pointers()
{
    int a = 5;
    int *pa = &a;
    int **pp = &pa;
    **pp = 3;
    // a == 3

    int b = 9;
    int *pb = &b;
    *pp = pb;
    **pp = 4;
    // b == 4
}
```



- mehrstufige Zeiger (z. B. Zeiger auf Zeiger) sind möglich
- Datentyp ist "Zeiger auf (Zieldatentyp)", z. B. "Zeiger auf Zeiger auf int"
- Umbiegen von Zeigern auf beliebiger Stufe ist möglich

Werte und Zeiger (4)

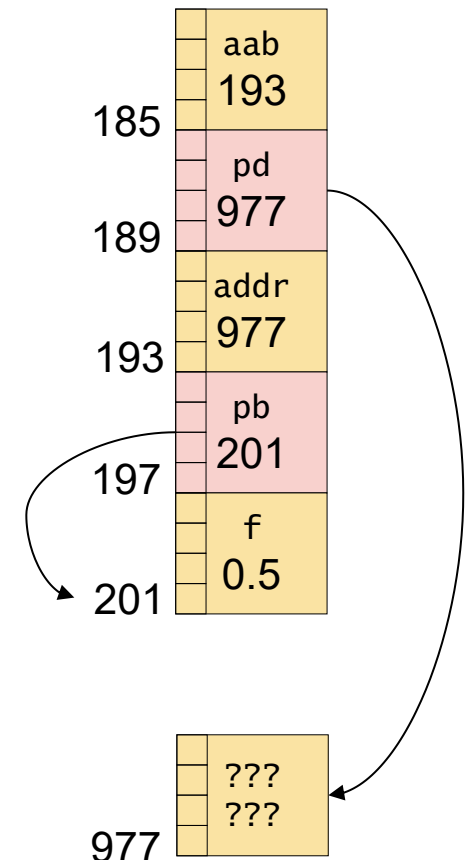
Typprüfungen auf Zeigertypen

```
void pointer_type_checks()
{
    float f = 0.5f;
    int *pa = &f;                // Kompilier-Fehler
    int *pb = (int*)&f;           // OK

    int addr = 977;
    float *pc = addr;             // Kompilier-Fehler
    float *pd = (float*)&addr;    // OK

    int addr_addr_a = &addr;      // Kompilier-Fehler
    int addr_addr_b = (int)&addr;  // OK
}
```

- Zeiger-Zuweisung wird auf Typ geprüft: $\text{int} \neq \text{int}^* \neq \text{int}^{**} \neq \text{float}^*$
- *Uminterpretation* mit C-style cast: $(\text{zieltyp})\text{quellvariable}$
- **Achtung:** Uminterpretation ist fehleranfällig!
(am Beispiel: pd zeigt auf selbstgewählte Adresse 977, was passiert, wenn man dort Daten hinschreibt?)



Werte und Zeiger (5)

Umgang mit kaputten Zeigern

```
void cause_npe()
{
    int *i = nullptr;
    *i = 4;                // Absturz?
}

int* get_local_ptr()
{
    int a = 5;
    return &a;
}

void cause_segfault()
{
    (*get_local_ptr()) = 3; // Absturz?
}

void another_segfault()
{
    *(int*)(100) = 5;       // Absturz?
}
```



Who is this General Fault,
and what is he doing
to my computer?

Beobachtung: Das Programm stürzt beim Zugriff auf das Ziel bestimmter Zeiger ab.

Werte und Zeiger (6)

Undefiniertes Verhalten

Motivation:

der Sprachstandard C++ definiert **gar nicht**, was in **bestimmten** Fehlerfällen passiert

→ korrekte C++-Programme dürfen diese Fehlerfälle niemals auslösen

→ da die Fehlerfälle nicht auftreten, können langsame Fehlerprüfungen entfallen

→ löst ein inkorrektes Programm einen solchen Fehlerfall trotzdem aus,
tritt das namensgebende *undefinierte Verhalten* auf

Beispiele:

- Zugriff aufs Ziel von `nullptr`-Zeiger
 - in C# / Java: `NullPointerException`
 - in C++: *undefiniertes Verhalten*
- Zugriff aufs Ziel von *ungültigem Zeiger*
(d. h. Zeiger ist weder `nullptr`, noch hat er gültige Zielvariable)
 - in C# / Java: solche Zeiger gibt es nicht
 - in C++: *undefiniertes Verhalten*

Werte und Zeiger (7)

Die Krux am undefinierten Verhalten

undefiniertes Verhalten kann sich z. B. so äußern:

- das Programm kompiliert nicht
- Absturz und Fehlermeldung
- Absturz... 10 Minuten später, an anderer Stelle
- fehlerhafte Daten tauchen in Datenbank auf
- alles funktioniert scheinbar korrekt...
- ... bis der Kunde das Programm laufen lässt
- ein anderes Programm stürzt ab
- Festplatte wird gelöscht
- das FBI wird informiert
- das verlorene Bernsteinzimmer erscheint

Undefiniertes Verhalten unbedingt vermeiden!

- Das ist gar nicht so leicht, weil es sehr viele Spezialfälle für undef. Vh. gibt!



[<https://de.wikipedia.org/wiki/Bernsteinzimmer>]

Werte und Zeiger (8)

Abgrenzung undefinierten Verhaltens

- Kann wirklich etwas "Zufälliges" / "Unerklärbares" in einem Computer passieren?

```
int i;           // uninitialized (random) number
std::cout << i;  // undefined behaviour, usually: output random number
```

- Ist die Ausgabe einer undefinierten `int`-Zahl undefiniertes Verhalten?
- **Exkurs:** C++ initialisiert eingebaute Variablen auf dem Stack nicht!

```
int n = random_number_from_radiation(100); // create randomly-sized array
for (int i = 0; i < n; ++i) array.insert(i); // unspecified behaviour
```

- Ist es undefiniertes Verhalten, wenn einer von 10 definierten Fällen auftritt?
- Ist es undefiniertes Verhalten, wenn einer von beliebig vielen Fällen auftritt?

begriffliche Trennung:

- undefiniertes Verhalten → es ist gar nichts festgelegt, z. B. Absturz möglich
- *unspezifiziertes Verhalten* → Programm folgt einem (von vielen) korrekten Abläufen

Vortragsübersicht

2 – Grundlagen C und C++

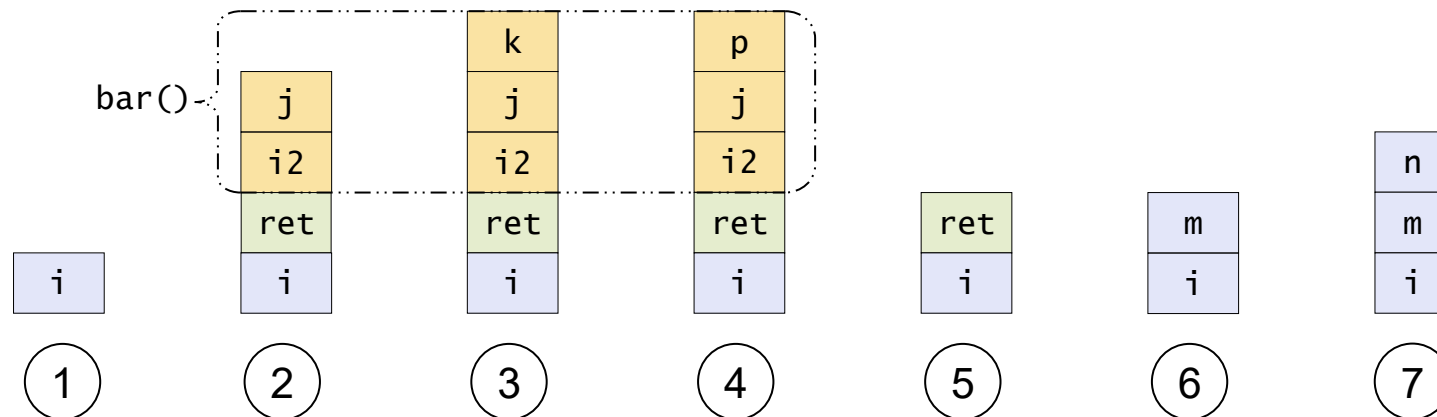
- C und C++ im Überblick
- Compiler-Pipeline
- Programmaufbau
- Werte und Zeiger
- Stack und Heap

Stack und Heap (1)

Variablen von eingebautem Typ auf dem Stack

```
void foo()
{
    int i; ①
    bar();
    for (int m=0; m<3; ++m) ⑥
        { int n; } ⑦
}
```

```
int bar()
{
    int i, j; ②
    { int k; } ③
    int p; ④
    return 10; ⑤
    int r; // not on stack
}
```



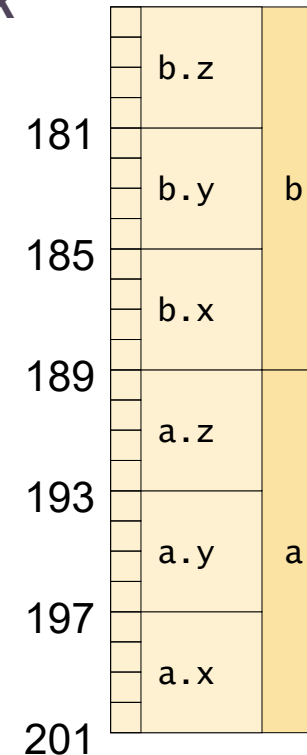
- *Stack* (dt. *Stapel*, *Halde*, *Keller*) kann nur oben Einträge erzeugen oder löschen
→ Stack im engeren Sinne erlaubt Zugriff nur oben, Laufzeitkeller kann überall zugreifen
- eingebaute Typen (`int`, `float`) lassen sich in C wie in C++ auf einen Stapel legen
- Variablen liegen direkt auf dem Stapel und werden am Bereichsende gelöscht

Stack und Heap (2)

Layout von zusammengesetzten Datentypen auf dem Stack

```
typedef struct point3f_  
{  
    float x;  
    float y;  
    float z;  
} point3f;  
  
void f()  
{  
    point3f a;  
    point3f b;  
    f(); // undefiniertes Verhalten  
}
```

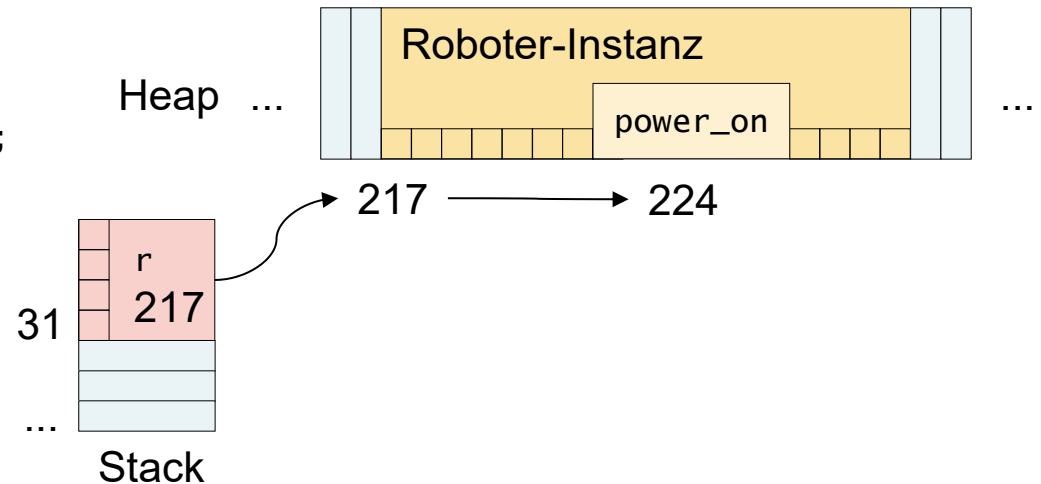
- Variablen von zusammengesetzten Datentypen auf dem Stack haben ihre (direkten) Member ebenfalls auf dem Stack liegen
- der Stack hat eine feste, begrenzte, vorab wählbare Größe (bei Desktop-OS üblicherweise wenige Megabyte)
→ **undefiniertes Verhalten, falls der Stack überläuft (z. B. wegen Rekursion)**



Stack und Heap (3)

Variablen auf dem Heap

```
typedef struct robot_  
{ bool power_on; /* ... */ } robot;  
void dancing_robot()  
{  
    robot *r = new robot();  
    r->power_on = true;  
  
    // oder alternativ:  
    (*r).power_on = true;  
}
```



- Anlegen einer Variablen auf dem Heap in C++ mittels `new`-Schlüsselwort
- Daten der Variablen liegen in konsekutiven Heap-Speicherzellen
- speichere erste Adresse der Heap-Variable in lokaler Stack-Variable von Zeigertyp
- Indirektion (mit `(*zeiger).` oder Kurzschreibweise `zeiger->`) greift auf Zielvariable zu

Achtung:

- Zwischen Zeiger `r` und Zielvariable `*r` unterscheiden!
- in der Praxis bei Bedarf bessere Nomenklatur nutzen, z. B. `r_ptr` statt `r`
→ ist aber stellenweise für die Lesbarkeit schlecht (Stichwort *ungarische Notation*)

Stack und Heap (4)

Lebenszeit auf dem Heap

```
void create_robot()
{
    robot *r = new robot();
    return;
}
```

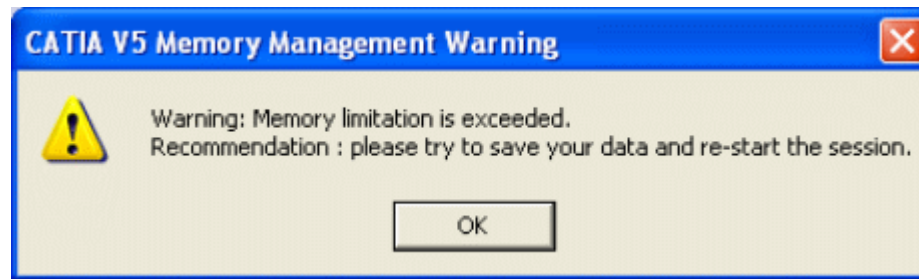
- der Zeiger *r* ist eine lokale Variable auf dem Stack und wird automatisch gelöscht, sobald die Funktion endet
- das Ziel von *r* wird nicht automatisch gelöscht, sondern bleibt auf dem Heap
- späteres explizites Löschen unmöglich, da kein Zeiger auf das ehemalige Ziel von *r* mehr existiert
→ **Speicherleck!**

Problem:

- Speicher steht nicht mehr für andere Daten zur Verfügung
→ das ist ein Bug, aber **kein** undefiniertes Verhalten!

Stack und Heap (5)

Speicherlecks in der Praxis



[<https://www.3ds.com>]

Stack und Heap (6)

Variablen vom Heap löschen

```
void dancing_robot()
{
    robot *r = new robot();
    r->power_on = true;
    delete r;
    r = nullptr; // passiert nicht automatisch, hier (!) unnötig
}
```

- Syntax in C++: `delete` zeigeraufvariable

Achtung:

- der Zeiger wird durch `delete` in der Regel nicht auf `nullptr` gesetzt
- man darf nur mit `new` angelegte Heap-Variable mit `delete` löschen
- man darf jede Variable nur exakt einmal löschen

Faustregel:

Wer `new` sagt, muss auch `delete` sagen!

Ausnahme: Automatisierung (nächster Vortrag)



[<http://4.bp.blogspot.com/s400/spiritual-down-the-drain.jpg>]

→ sonst undef. Vh.

→ sonst undef. Vh.

Stack und Heap (7)

Zugriff nach Variablenlöschung

```
void shoot_the_bartender(robot *bartender)
{
    // Objekt *bartender wird hier gelöscht,
    // aber es gibt noch Zeiger darauf.
    delete bartender;
}

void robot_saloon()
{
    robot *bartender = new robot();
    bartender->sing = true;
    shoot_the_bartender(bartender);

    // undefiniertes Verhalten:
    // Zugriff auf bereits gelöscht *bartender
    bartender->make_drinks = true;
}
```



- **undefiniertes Verhalten** bei Zugriff nach dem Löschen
(dazu gehört auch eine mehrfache Löschung!)
- übliche Äußerung: Absturz, z. B. segmentation fault

[<http://imgc.allpostersimages.com/images/frank-cotham-two-cowboys-sitting-outside-a-saloon-with-remote-controls-watch-toy-robot-new-yorker-cartoon.jpg>]

Stack und Heap (8)

Wann Variable auf dem Stack?

Standardfall!

Beispiele:

- Variable wird nur lokal benötigt (Zähler-Variablen, temporäre Variablen)
- Rückgabe einer Variablenkopie ist kostenarm möglich
- kein Zeiger auf identische Variable außerhalb des lokalen Bereichs benötigt
- Variable ist klein genug, um keinen Stapelüberlauf zu verursachen

Wann Variable auf dem Heap?

falls Variable nicht auf dem Stack platziert werden kann

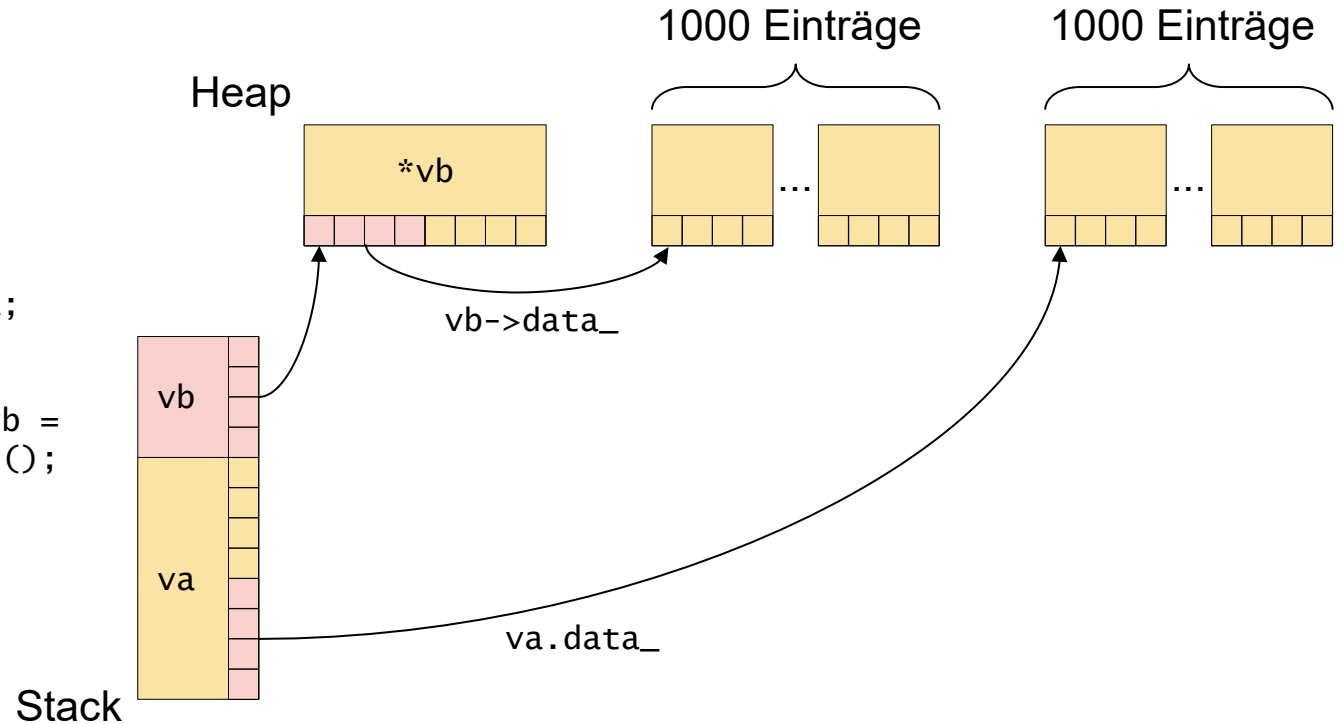
Beispiele:

- Variable ist sehr groß (pauschal > 10kB)
- Variablenidentität muss viele Funktionsaufrufe überleben
- Variablenkopie ist teuer oder nicht möglich (z. B. Bild- oder Datenbankobjekt)
- Variable ist semantisch bedeutend (z. B. Ressourcenmanager)

Stack und Heap (9)

Ausblick Heap und Standardcontainer

```
class std::vector<int> // Pseudocode!  
{  
    /* ... */  
    int* data_;  
    int size_;  
};  
  
void f()  
{  
    std::vector<int> va;  
    a.resize(1000);  
  
    std::vector<int>* vb =  
        new vector<int>();  
    b->resize(1000);  
    delete vb;  
}
```



Achtung:

- viele Container (z. B. `std::vector` und `std::string` aus der Standardbibliothek) lagern ihre Nutzdaten ohnehin (per Zeiger als Member) auf den Heap aus
→ eine Doppelung ist hier (z. B. wegen Performance) ungünstig!

Zusammenfassung

Grundlagen der C- und C++-Programmierung

- C und C++ sind effiziente Sprachen, weil sie Prüfungen weglassen
→ undefiniertes Verhalten als Folge
- C++ ist C und mehr, speziell: mehr Stolpersteine für undef. Vh.
- Compiler-Pipeline umfasst Quellcode- und Header-Dateien
- textuelles Einbinden von Headern durch den Präprozessor
- eindeutig benannte Inclusion Guards verhindern Mehrfacheinbindung
- Stack und Heap zur Lebenszeitverwaltung
- Zeiger als Datentyp bzw. als Variable:
Deklaration, Dereferenzierung, Typprüfungen, mehrstufige Zeiger, ungültige Zeiger
- Heap-Speicherverwaltung in C++ mittels `new` und `delete`
→ jede mit `new` angelegte Variable exakt einmal mit `delete` löschen